



深圳大学
Shenzhen University

互联网编程

第7章

URLConnection

深圳大学计算机与软件学院

主要内容

- 1. URLConnection简介
- 2. 读取服务器数据
- 3. 缓存
- 4. 连接与请求配置
- 5. 向服务器写入数据
- 6. HttpURLConnection



1.URLConnection简介

- public abstract class URLConnection extends Object
- URLConnection是一个抽象类
- URLConnection -> HttpURLConnection
- 提供了更多与HTTP服务器的交互控制
- 可以检查服务器发送的首部，做出响应。
- 可以设置客户端请求中使用的首部字段。
- 可用GET、POST、PUT等HTTP请求方法向服务器发送数据。



URLConnection类的使用步骤

直接使用URLConnection类的程序基本编写步骤：

1. 创建URL对象
2. 调用URL.openConnection()获取URLConnection对象
3. 配置URLConnection对象
4. 读取头信息
5. 获得输入流，并读数据
6. 获得输出流，并写数据
7. 关闭连接



创建URLConnection对象

- URLConnection类仅有一个为保持类型的构造函数：
`protected URLConnection(URL url)`
- 例如要编写一个协议处理器需派生URLConnection的子类才会使用此构造函数
- 通常使用该类的openConnection方法来创建对象

```
try {  
    URL u = new URL("http://www.szu.edu.cn/");  
    URLConnection uc = u.openConnection();  
    // read from the URL...  
}  
  
catch (MalformedURLException ex) {  
    System.err.println(ex);  
}  
  
catch (IOException ex) {  
    System.err.println(ex);  
}
```



2. 读取服务器的数据

- 使用一个URLConnection对象从一个URL获取数据，编程基本步骤：
 - (1) 构造一个URL对象
 - (2) 调用这个URL对象的openConnection()方法获取此URL的URLConnection对象
 - (3) 调用这个URL connection的getInputStream()方法
 - (4) 使用通常的流API读取输入流
- URL connection的getInputStream()方法返回一个通用InputStream，可以读取和解析服务器发送的数据。



读取服务器的数据

- 例子:

- ▣ 使用URLConnection getInputStream()方法下载一个web页面。



```
InputStream in = null;
try {
    // Open the URL for reading
    URL u = new URL(args[0]);
    URLConnection uc = u.openConnection();
    in = uc.getInputStream();
    // buffer the input to increase performance
    in = new BufferedInputStream(in);
    // chain the InputStream to a Reader
    Reader r = new InputStreamReader(in);
    int c;
    while ((c = r.read()) != -1) {
        System.out.print((char) c);
    }
} catch (MalformedURLException ex) {
    System.err.println(args[0] + " is not a parseable URL");
} catch (IOException ex) {
    System.err.println(ex);
} finally {
    if (in != null) {
        try {
            in.close();
        } catch (IOException e) {
            // ignore
        }
    }
}
```


读取服务器的数据

演示ch7. SourceViewer2完整代码
实现使用URLConnection下载一个Web页面

SourceViewer2, 使用URLConnection

SourceViewer2.java

```
3 import java.io.*;
4 import java.net.*;
5 public class SourceViewer2 {
6     public static void main (String[] args) {
7         if (args.length > 0) {
8             try {
9                 // Open the URLConnection for reading
10                URL u = new URL(args[0]);
11                URLConnection uc = u.openConnection();
12                try (InputStream raw = uc.getInputStream()) { // autoclose
13                    InputStream buffer = new BufferedInputStream(raw);
14                    // chain the InputStream to a Reader
15                    Reader reader = new InputStreamReader(buffer);
16                    int c;
17                    while ((c = reader.read()) != -1) {
18                        System.out.print((char) c);
19                    }
20                }
21            } catch (MalformedURLException ex) {
22                System.err.println(args[0] + " is not a parseable URL");
23            } catch (IOException ex) {
24                System.err.println(ex);
25            }
26        }
27    }
28 }
```



```
InputStream in = null;
try {
    // Open the URL for reading
    URL u = new URL(args[0]);
    in = u.openStream();
    // buffer the input to increase performance
    in = new BufferedInputStream(in);
    // chain the InputStream to a Reader
    Reader r = new InputStreamReader(in);
    int c;
    while ((c = r.read()) != -1) {
        System.out.print((char) c);
    }
} catch (MalformedURLException ex) {
    System.err.println(args[0] + " is not a parseable URL");
} catch (IOException ex) {
    System.err.println(ex);
} finally {
    if (in != null) {
        try {
            in.close();
        } catch (IOException e) {
            // ignore
        }
    }
}
```

对比Ch5.SourceViewer使用URL类

URL与URLConnection区别

- URLConnection提供了对HTTP头信息访问
- URLConnection可以配置发给服务器的请求参数
- URLConnection除了可读取服务器数据外，还可向服务器写入数据



读取HTTP首部

- 一个HTTP服务器响应的头部信息头部中的常用字段有：

- Content-type
- Content-length
- Content-encoding
- Date
- Last-modified
- Expires

一个**Apache Web**服务器返回的响应信息中的**HTTP头部**：

HTTP/1.1 301 Moved Permanently
Date: Sun, 21 Apr 2013 15:12:46 GMT
Server: Apache
Location: <http://www.ibiblio.org/>
Content-Length: 296
Connection: close
Content-type: text/html; charset = iso-8859-1



头字段信息获取的方法（2个）

■ `public String getContentType()`

- 返回响应主体的MIME类型，包括字符编码方式，如：
`text/html; charset=UTF-8`
- 使得编程人员可以使用HTTP头部中指定的编码方式对文档解码。如果头部中没有指定编码方式，就使用ISO-8859-1，这是HTTP的默认编码方式。

■ `public int getContentLength()`

■ `public long getContentLengthLong()`

- 返回响应主体的字节数
- 如无Content-length首部，返回-1
- 当程序中需要准备知道要读取的字节数，或需要预先创建一个足够大的缓冲区来保存数据时使用。



```
try {  
    // set default encoding  
    String encoding = "ISO-8859-1";  
    URL u = new URL(args[0]);  
    URLConnection uc = u.openConnection();  
    String contentType = uc.getContentType();  
    int encodingStart = contentType.indexOf(" charset=");  
    if (encodingStart != -1) {  
        encoding = contentType.substring(encodingStart + 8);  
    }  
    InputStream in = new BufferedInputStream(uc.getInputStream());  
    //Reader r = new InputStreamReader(in, encoding);  
    Reader r = new InputStreamReader(in);  
    int c;  
    while ((c = r.read()) != -1) {  
        System.out.print((char) c);  
    }  
    r.close();  
  
    System.out.println(" charset=" + encoding);  
}  
catch (MalformedURLException ex) {  
    System.err.println(args[0] + " is not a parseable URL");  
}  
catch (UnsupportedEncodingException ex) {  
    System.err.println("Server sent an encoding Java does not support: " + ex.getMessage());  
}  
catch (IOException ex) {  
    System.err.println(ex);  
}
```

使用HTTP头部中指定的编码方式对文档解码
改进ex7-1SourceViewer2。

EncodingAwareSourceViewer.java

```
3 import java.io.*;
4 import java.net.*;
5 public class EncodingAwareSourceViewer {
6     public static void main (String[] args) {
7         for (int i = 0; i < args.length; i++) {
8             try {
9                 String encoding = "ISO-8859-1"; // set default encoding
10                 URL u = new URL(args[i]);
11                 URLConnection uc = u.openConnection();
12                 String contentType = uc.getContentType();
13                 int encodingStart = contentType.indexOf("charset=");
14                 if (encodingStart != -1) {
15                     encoding = contentType.substring(encodingStart + 8); // 设置正确的编码
16                 }
17                 InputStream in = new BufferedInputStream(uc.getInputStream());
18                 Reader r = new InputStreamReader(in, encoding);
19                 int c;
20                 while ((c = r.read()) != -1) {
21                     System.out.print((char) c);
22                 }
23                 r.close();
24             } catch (MalformedURLException ex) {
25                 System.err.println(args[0] + " is not a parseable URL");
26             } catch (UnsupportedEncodingException ex) {
27                 System.err.println(
28                     "Server sent an encoding Java does not support: " + ex.getMessage());
29             } catch (IOException ex) {
30                 System.err.println(ex);
31             }
32         }
33     }
34 }
```

演示ch7. EncodingAwareSourceViewer, 用
正确的字符集下载一个Web页面。

找出服务器的编码

控制编码格式

```
public static void saveBinaryFile(URL u) throws IOException {
    URLConnection uc = u.openConnection();
    String contentType = uc.getContentType();
    int contentLength = uc.getContentLength();
    if (contentType.startsWith("text/") || contentLength == -1 ) {
        throw new IOException("This is not a binary file.");
    }
    try (InputStream raw = uc.getInputStream()) {
        InputStream in = new BufferedInputStream(raw);
        byte[] data = new byte[contentLength];
        int offset = 0;
        while (offset < contentLength) {
            int bytesRead = in.read(data, offset, data.length - offset);
            if (bytesRead == -1) break;
            offset += bytesRead;
        }
        if (offset != contentLength) {
            throw new IOException("Only read " + offset + " bytes; Expected " + contentLength + " bytes");
        }
        String filename = u.getFile();
        filename = filename.substring(filename.lastIndexOf('/') + 1);
        try (FileOutputStream fout = new FileOutputStream(filename)) {
            fout.write(data);
            fout.flush();
        }
    }
}
```

使用`getContentLength()`方法例子。
正确可靠从HTTP服务器下载一个二进制文件的例子。

```
3 import java.io.*;
4 import java.net.*;
5 public class BinarySaver {
6
7     public static void main (String[] args) {
8         for (int i = 0; i < args.length; i++) {
9             try {
10                 URL root = new URL(args[i]);
11                 saveBinaryFile(root);
12             } catch (MalformedURLException ex) {
13                 System.err.println(args[i] + " is not URL I understand.");
14             } catch (IOException ex) {
15                 System.err.println(ex);
16             }
17         }
18     }
19
20     public static void saveBinaryFile(URL u) throws IOException {
21         URLConnection uc = u.openConnection();
22         String contentType = uc.getContentType();
23         int contentLength = uc.getContentLength();
24         if (contentType.startsWith("text/") || contentLength == -1 ) {
25             throw new IOException("This is not a binary file.");
26         }
27     }
28 }
```

```
27
28     try (InputStream raw = uc.getInputStream()) {
29         InputStream in = new BufferedInputStream(raw);
30         byte[] data = new byte[contentLength];
31         int offset = 0;
32         while (offset < contentLength) {
33             int bytesRead = in.read(data, offset, data.length - offset);
34             if (bytesRead == -1) break;
35             offset += bytesRead;
36         }
37         if (offset != contentLength) {
38             throw new IOException("Only read " + offset
39                 + " bytes; Expected " + contentLength + " bytes");
40         }
41         String filename = u.getFile();
42         filename = filename.substring(filename.lastIndexOf('/') + 1);
43         try (FileOutputStream fout = new FileOutputStream(filename)) {
44             fout.write(data);
45             fout.flush();
46         }
47     }
48 }
49 }
```

演示ch7. BinarySaver，从Web网站下载二进制文件并保存到磁盘。

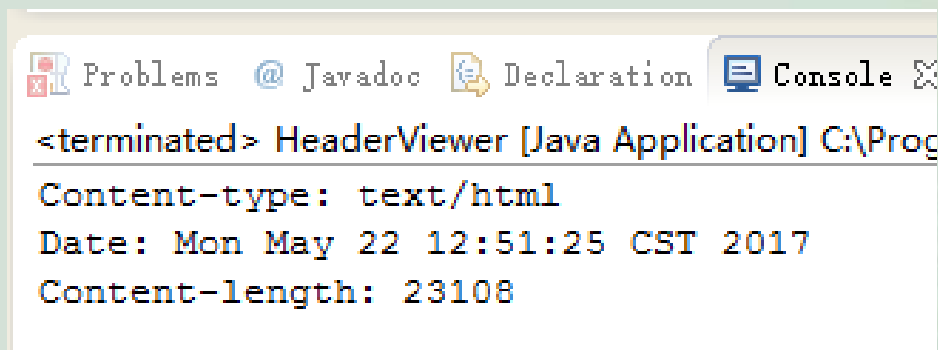
头字段信息获取的方法（4个）

- `public String getContentEncoding()`
 - ▣ 返回内容压缩编码（与字符编码不同）
- `public long getDate()`
 - ▣ 发送文档的时间
- `public long getExpiration()`
 - ▣ 文档失效时间
- `public long getLastModified()`
 - ▣ 文档最后修改时间



头部方法使用例子

- 代码例子ch7. HeaderViewer
- 从命令行读取url，使用访问头部信息的6个方法，显示内容类型、内容长度、内容编码方式、最后修改日期、过期日期和当前日期。
- 运行 HeaderViewer,输入 <http://www.szu.edu.cn/szu.asp>参数的运行结果



```
Problems  @ Javadoc  Declaration  Console  X
<terminated> HeaderViewer [Java Application] C:\Prog
Content-type: text/html
Date: Mon May 22 12:51:25 CST 2017
Content-length: 23108
```



HeaderViewer.java


```
2
3 import java.io.*;
4 import java.net.*;
5 import java.util.*;
6 public class HeaderViewer {
7
8     public static void main(String[] args) {
9         for (int i = 0; i < args.length; i++) {
10             try {
11                 URL u = new URL(args[0]);
12                 URLConnection uc = u.openConnection();
13                 System.out.println("Content-type: " + uc.getContentType());
14                 if (uc.getContentEncoding() != null) {
15                     System.out.println("Content-encoding: "
16                                     + uc.getContentEncoding());
17                 }
18                 if (uc.getDate() != 0) {
19                     System.out.println("Date: " + new Date(uc.getDate()));
20                 }
21                 if (uc.getLastModified() != 0) {
22                     System.out.println("Last modified: "
23                                     + new Date(uc.getLastModified()));
24                 }
25                 if (uc.getExpiration() != 0) {
26                     System.out.println("Expiration date: "
27                                     + new Date(uc.getExpiration()));
28                 }
19
```

HeaderView.java

```
29     if (uc.getContentLength() != -1) {
30         System.out.println("Content-length: " + uc.getContentLength());
31     }
32 } catch (MalformedURLException ex) {
33     System.err.println(args[i] + " is not a URL I understand");
34 } catch (IOException ex) {
35     System.err.println(ex);
36 }
37 System.out.println();
38 }
39 }
40 }
```

HeaderView.java



获取任意头字段方法

json格式访问

- `public String getHeaderField(String name)`
- `String contentType = uc.getHeaderField("content-type");`
- `String contentEncoding = uc.getHeaderField("content-encoding");`
- `String data = uc.getHeaderField("date");`
- `String expires = uc.getHeaderField("expires");`
- `String contentLength = uc.getHeaderField("Content-length");`



使用获取任意首部字段方法显示整个HTTP请求

- 编程实现：提供任一个url，在控制台显示输出HTTP服务器响应整个头部。
- 演示例子代码ch7. AllHeaders.java
- 输入参数 `http://www.szu.edu.cn/szu.asp`，运行结果：

```
<terminated> AllHeaders [Java Application] C:\Program Files\Java\jdk1.7.0_07\bin\javaw.exe
Date: Thu, 25 May 2017 08:59:14 GMT
Server: Microsoft-IIS/6.0
X-Powered-By: ASP.NET
Content-Length: 23091
Content-Type: text/html
Set-Cookie: ASPSESSIONIDACSCRSQD=LFGJDLEDLIMJMLCKIODDDALC; path=/
Cache-control: private
```

不同服务器可能有不同的首部集。

```
3 import java.io.*;
4 import java.net.*;
5
6 public class AllHeaders {
7
8     public static void main(String[] args) {
9         for (int i = 0; i < args.length; i++) {
10             try {
11                 URL u = new URL(args[i]);
12                 URLConnection uc = u.openConnection();
13                 for (int j = 1; ; j++) {
14                     String header = uc.getHeaderField(j);
15                     if (header == null) break;
16                     System.out.println(uc.getHeaderFieldKey(j) + ": " + header);
17                 }
18             } catch (MalformedURLException ex) {
19                 System.err.println(args[i] + " is not a URL I understand.");
20             } catch (IOException ex) {
21                 System.err.println(ex);
22             }
23             System.out.println();
24         }
25     }
26 }
```

获得任意头字段的方法

- `public String getHeaderFieldKey(int n)`

- 第n个字段名

- `public String getHeaderField(int n)`

- 第n个字段值

- 这两个方法可以实现对所有头字段的遍历

- （因为无法预知HTTP服务器会发送哪些字段）



3.Web缓存

- 为啥需要缓存？
- 比如Web浏览器从远程HTTP服务器请求加载一次一个页面和图片后，保存在本地缓存中，每次需要时从缓存加载，不是每次都从远程服务器加载。
- 默认：使用GET通过HTTP访问的页面可以缓存，也应缓存。使用HTTPS或POST访问的页面不应缓存。
- 通过检查和使用HTTP响应头部(Expires和Cache-Control)可以控制缓存
- Expires：指示可以缓存这个资源的表示，直到指定的时间为止。
- Cache-Control:提供了细粒度的缓冲策略。（课后熟悉P198页各属性的含义）

Web缓存

- 你的HTTP客户端程序查看到下面这个HTTP响应头后，且本地缓存中有一个Etag为67099097696afcf1b67e的资源，是否还需加载整个文档？

一个HTTP响应头部：

```
HTTP/1.1 200 OK
Date: Sun, 21 Apr 2013 15:12:46 GMT
Server: Apache
Connection: close
Content-type: text/html; charset = iso-8859-1
Cache-control: max-age=604800
Expires: Sun,28 Apr 2013 15:12:46GMT
Last-modified: Sat, 20 Apr 2013 09:55:04 GMT
Etag: "67099097696afcf1b67e"
```

资源可以缓存604800秒，即缓存到一周以后。
缓存中已经有一个更新的副本。
无需加载整个文档。

- HTTP客户端检查Cache-control头部，可以利用信息：
 - 1.若本地缓存有此资源的一个表示，且未过期，可直接使用此资源，无需与服务器交互。
 - 2.若本地缓存有此资源的一个表示，且已过期，在完成完整GET之前，可检查服务器的HEAD首部，查看资源是否已改变。

```
2 import java.util.Date;
3 import java.util.Locale;
4 public class CacheControl {
5
6     private Date maxAge = null;
7     private Date sMaxAge = null;
8     private boolean mustRevalidate = false;
9     private boolean noCache = false;
10    private boolean noStore = false;
11    private boolean proxyRevalidate = false;
12    private boolean publicCache = false;
13    private boolean privateCache = false;
14
15    public CacheControl(String s) {
16        if (s == null || !s.contains(":")) {
17            return; // default policy
18        }
19
20        String value = s.split(":")[1].trim();
21        String[] components = value.split(",");
22
```

演示讲解一个用于解析和查询
Cache-control首部的java类
Ch7.CacheControl.java。

```
23     Date now = new Date();
24     for (String component : components) {
25         try {
26             component = component.trim().toLowerCase(Locale.US);
27             if (component.startsWith("max-age=")) {
28                 int secondsInTheFuture = Integer.parseInt(component.substring(8));
29                 maxAge = new Date(now.getTime() + 1000 * secondsInTheFuture);
30             } else if (component.startsWith("s-maxage=")) {
31                 int secondsInTheFuture = Integer.parseInt(component.substring(8));
32                 sMaxAge = new Date(now.getTime() + 1000 * secondsInTheFuture);
33             } else if (component.equals("must-revalidate")) {
34                 mustRevalidate = true;
35             } else if (component.equals("proxy-revalidate")) {
36                 proxyRevalidate = true;
37             } else if (component.equals("no-cache")) {
38                 noCache = true;
39             } else if (component.equals("public")) {
40                 publicCache = true;
41             } else if (component.equals("private")) {
42                 privateCache = true;
43             }
44         } catch (RuntimeException ex) {
45             continue;
46         }
47     }
48 }
49
```

```
49
50 public Date getMaxAge() {
51     return maxAge;
52 }
53 public Date getSharedMaxAge() {
54     return sMaxAge;
55 }
56 public boolean mustRevalidate() {
57     return mustRevalidate;
58 }
59 public boolean proxyRevalidate() {
60     return proxyRevalidate;
61 }
62 public boolean noStore() {
63     return noStore;
64 }
65 public boolean noCache() {
66     return noCache;
67 }
68 public boolean publicCache() {
69     return publicCache;
70 }
71 public boolean privateCache() {
72     return privateCache;
73 }
74 }
```



Java的Web缓存

- JVM只支持一个共享缓存，在HTTP客户端程序中要使用缓存需要使用到自定义的3个具体子类来安装URL类使用的系统级缓存。
 - ResponseCache的一个具体子类
 - CacheRequest的一个具体子类
 - CacheResponse的一个具体子类
- 使用ResponseCache.setDefault()方法把自定义缓存子类对象安装为默认缓存，来处理自定义的CacheRequest和CacheResponse子类。



Java的Web缓存-- CacheRequest类

- ResponseCache的put()方法返回一个CacheRequest类型的对象。
- CacheRequest类表示在 ResponseCache 中存储资源的通道。
- CacheRequest类的实例包装了一个OutputStream对象，HTTP客户端程序可以调用该对象来将资源数据存储在缓存中。
- CacheRequest类 ex7-7 CacheRequest.java :

Package java.net

```
Public abstract class CacheRequest {  
    //返回可以将响应正文写入cache库的 OutputStream  
    public abstract OutputStream getBody( ) throws IOException;  
    public abstract void abort(); //中止缓存响应的尝试  
}
```

自定义CacheRequest子类

- 自定义一个CacheRequest子类，可返回一个ByteArrayOutputStream，支持ResponseCache对象的put()方法将资源数据以字节数组文件存入缓存中。
- 演示子类代码例子ch7. SimpleCacheRequest.java

```
SimpleCacheRequest.java ✕
3 import java.io.*;
4 import java.net.*;
5
6 public class SimpleCacheRequest extends CacheRequest {
7
8     private ByteArrayOutputStream out = new ByteArrayOutputStream();
9
10    @Override
11    public OutputStream getBody() throws IOException {
12        return out;
13    }
14
15    @Override
16    public void abort() {
17        out.reset();
18    }
19
20    public byte[] getData() {
21        if (out.size() == 0) return null;
22        else return out.toByteArray();
23    }
24 }
```



Java的Web缓存-- CacheResponse类

- ResponseCache的get()方法从缓存中获取资源的数据和首部，包装在一个CacheResponse类型的对象中返回。
- CacheResponse类表示在 ResponseCache 中检索资源的通道。
- CacheResponse类的实例包装了一个URI请求资源的数据和首部，HTTP客户端程序可以调用该对象将资源的数据或首部从缓存中取出。
- CacheResponse类 ex7-9 CacheResponse:

Package java.net

```
Public abstract class CacheResponse{  
    //返回响应的主体作为InputStream  
    public abstract Map<String, List<String>> getHeaders() throws IOException;  
    public abstract InputStream getBody() throws IOException; //返回响应头作为一个Map}
```

自定义CacheResponse子类

- 自定义一个CacheResponse子类，与一个自定义的CacheRequest子类（SimpleCacheRequest）对象和一个CacheControl对象绑定，支持ResponseCache对象的get()方法从缓存中读取资源的数据或首部。
- 演示一个自定义CacheResponse子类代码例子ch7.SimpleCacheResponse。

```
SimpleCacheResponse.java ✕
3 import java.io.*;
4 import java.net.*;
5 import java.util.*;
6 public class SimpleCacheResponse extends CacheResponse {
7
8     private final Map<String, List<String>> headers;
9     private final SimpleCacheRequest request;
10    private final Date expires;
11    private final CacheControl control;
12
13    public SimpleCacheResponse(
14        SimpleCacheRequest request, URLConnection uc, CacheControl control)
15        throws IOException {
16
17        this.request = request;
18        this.control = control;
19        this.expires = new Date(uc.getExpiration());
20        this.headers = Collections.unmodifiableMap(uc.getHeaderFields());
21    }
```

```
23 @Override
24 public InputStream getBody() {
25     return new ByteArrayInputStream(request.getData());
26 }
27
28 @Override
29 public Map<String, List<String>> getHeaders()
30     throws IOException {
31     return headers;
32 }
33
34 public CacheControl getControl() {
35     return control;
36 }
37
38 public boolean isExpired() {
39     Date now = new Date();
40     if (control.getMaxAge().before(now)) return true;
41     else if (expires != null && control.getMaxAge() != null) {
42         return expires.before(now);
43     } else {
44         return false;
45     }
46 }
47 }
```

Java的Web缓存-- ResponseCache类

- 代表URLConnection缓存的实现。
- 它的实例可以通过
`ResponseCache.setDefault(ResponseCache)`向系统注册,系统将调用这个对象:
- (1) 存储从外部源已检索资源数据到缓存中。
- (2) 试图获取可能是存储在缓存中的所请求的资源。
- ResponseCache实现决定哪些资源应该被缓存,多长时间。如果不能从缓存检索请求的资源,协议处理程序将从其原始位置获取资源。
- ResponseCache类 如下 :



Java的Web缓存-- ResponseCache类

Package java.net

```
Public abstract class ResponseCache extends Object {  
    //检索缓存的响应基于请求uri,请求方法和请求头。  
    public abstract CacheResponse get(URI uri, String rqstMethod,  
    Map<String,List<String>> rqstHeaders)  
        throws IOException;  
    //协议处理器调用此方法检索资源之后,和ResponseCache必须决定  
    //是否存储缓存的资源。  
    public abstract CacheRequest put(URI uri, URLConnection conn)  
        throws IOException;  
    ... ..  
}
```



自定义ResponseCache子类

- 自定义一个ResponseCache子类，使用一个大的线程安全HashMap将有限数量的响应存储在内存中。
- 演示自定义ResponseCache子类代码例子ex7-11 ch7.MemoryCache。

```
MemoryCache.java ✕
4 import java.net.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7
8 public class MemoryCache extends ResponseCache {
9
10     private final Map<URI, SimpleCacheResponse> responses
11         = new ConcurrentHashMap<URI, SimpleCacheResponse>();
12     private final int maxEntries;
13
14     public MemoryCache() {
15         this(100);
16     }
17
18     public MemoryCache(int maxEntries) {
19         this.maxEntries = maxEntries;
20     }
```

```
21
22 @Override
23 public CacheRequest put(Uri uri, HttpURLConnection conn)
24     throws IOException {
25
26     if (responses.size() >= maxEntries) return null;
27
28     CacheControl control = new CacheControl(conn.getHeaderField("Cache-Control"));
29     if (control.noStore()) {
30         return null;
31     } else if (!conn.getHeaderField(0).startsWith("GET ")) {
32         // only cache GET
33         return null;
34     }
35
36     SimpleCacheRequest request = new SimpleCacheRequest();
37     SimpleCacheResponse response = new SimpleCacheResponse(request, conn, control);
38
39     responses.put(uri, response);
40     return request;
41 }
```



```
42
43 @Override
44 public CacheResponse get(Uri uri, String requestMethod,
45     Map<String, List<String>> requestHeaders)
46     throws IOException {
47
48     if ("GET".equals(requestMethod)) {
49         SimpleCacheResponse response = responses.get(uri);
50         // check expiration date
51         if (response != null && response.isExpired()) {
52             responses.remove(response);
53             response = null;
54         }
55         return response;
56     } else {
57         return null;
58     }
59 }
60 }
```

4.配置URLConnection连接

- 抽象类 `URLConnection` 是一个超类，代表应用程序和 URL 之间的通信链接。
- 此类的实例可用于读取和写入此 URL 引用的资源。创建一个到 URL 的连接需要4个步骤：
 - (1) 通过在 URL 上调用 `openConnection` 方法创建连接对象。
 - (2) 处理设置参数和一般请求属性。
 - (3) 使用 `connect` 方法建立到远程对象的实际连接。
 - (4) 远程对象变为可用。远程对象的头字段和内容变为可访问。
- `URLConnection`类有7个受保护的实例字段，通过对这些字段的访问和修改可以配置和定义客户端如何向服务器做出请求。这些字段的访问和修改需使用字段对应的`get`和`set`方法。



配置URLConnection连接

字段摘要	
protected boolean	<u>allowUserInteraction</u> 如果为 true, 则在允许用户交互 (例如弹出一个验证对话框) 的上下文中对此 URL 进行检查。
protected boolean	<u>connected</u> 如果为 false, 则此连接对象尚未创建到指定 URL 的通信链接。
protected boolean	<u>doInput</u> 此变量由 setDoInput 方法设置。
protected boolean	<u>doOutput</u> 此变量由 setDoOutput 方法设置。
protected long	<u>ifModifiedSince</u> 有些协议支持跳过对象获取, 除非该对象在某个特定时间点之后又进行了修改。
protected <u>URL</u>	<u>url</u> URL 表示此连接要在互联网上打开的远程对象。
protected boolean	<u>useCaches</u> 如果为 true, 则只要有条件就允许协议使用缓存。

doOutput=true;表示除了通过这个URLConnection读取数据外, 还可以将数据写入到服务器。

useCaches=false;表示连接会绕过所有本地缓存, 重新从服务器下载文件。

Protected URL url 字段

- url字段指定了这个URLConnection连接的URL。
- 构造函数会在创建URLConnection时设置这个字段，此后不能再改变
- 可调用getURL()获取该字段的值
- Ex7-12 URLPrinter.java，打开一个指向<http://www.oreilly.com/>的URLConnection，获取这连接的URL，并显示。

URLPrinter.java

演示ch7.URLPrinter.java

```
3 import java.io.*;
4 import java.net.*;
5
6 public class URLPrinter {
7
8     public static void main(String[] args) {
9         try {
10             URL u = new URL("http://www.oreilly.com/"); //重点句，u对象存在期间不能再改变
11             URLConnection uc = u.openConnection();
12             System.out.println(uc.getURL()); //重点句
13         } catch (IOException ex) {
14             System.err.println(ex);
15         }
16     }
17 }
```

Protected boolean connected



- 当连接已经打开，connected为true。由任何导致URLConnection连接的方法设置
- 当连接关闭，connected 为false
- 没有直接读取或改变connected值的方法
- 该变量只能由java.net.URLConnection及其子类的实例访问。
- 注意：派生URLConnection子类编写一个协议处理器时，要正确设置该字段的值，否则程序可能出现严重的且很难诊断的bug



Protected boolean allowUserInteraction

- 该字段指示是否允许用户交互，默认值是false
- 当连接关闭，connected 为false
- 只能通过该字段对应的get/set方法访问
- 只能在URLConnection连接前设置，否则会抛出IllegalStateException异常

```
URL u=new  
URL("http://www.example.com/passwordProtectedPage.html");  
URLConnection uc = u.openConnection();//获得连接对象  
Uc.setAllowUserInteraction(true);  
InputStream in uc.getInputStream();//建立输入连接
```



Protected boolean doInput

- 该字段指示URLConnection可否用于写入服务器
- doInput为true表示连接写用于读/写服务器
- doInput为false表示连接只能用于读取服务器。
- 默认值是true
- 该字段的访问使用对应的get/set方法

```
Try{  
    URL u=new URL("http://www.oreilly.com");  
    URLConnection uc = u.openConnection();//获得连接对象  
    if(!uc.getDoInput()){  
        uc.setDoInput(true);  
    }  
    //从连接读取...  
} catch(IOException ex){  
    System.out.println(ex);  
}
```



Protected boolean doOutput

- 该字段指示程序可否使用URLConnection将输出发回服务器
- doOutput为true表示连接写用于写数据到服务器
- doOutput为false表示连接不能用于写数据到服务器。
- 默认值是false
- 该字段的访问使用对应的get/set方法

```
Try{
    URL u=new URL("http://www.oreilly.com");
    URLConnection uc = u.openConnection();//获得连接对象
    if(!uc.getDoOutput()){
        uc.setDoOutput(true);
    }
    //写入连接...
} catch(IOException ex){
    System.out.println(ex);
}
```



Protected boolean ifModifiedSince

- 该字段指示放置在URLConnection连接请求头的If-Modified-Since字段中的日期

客户端

服务器

客户端的设定时间后修改，服务器才会传回文档，否则报304错误码

服务器考虑If-Modified-Since字段，如果请求的文档在这个时间后有修改，服务器就发送此文档，否则不发送此文档，回复“未修改”消息。客户端从本地缓存中加载此文档。

```
GET / HTTP/1.1
Host: login.ibiblio.org:56452
Accept: text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2
Connection: close
If-Modified-Since: Fri, 31 Oct 2014 19:22:07 GMT
```

```
HTTP/1.0 304 Not Modified
Server: WN/1.15.1
Date: Sun, 02 Nov 2014 16:26:16 GMT
Last-modified: Fri, 29 Oct 2004 23:40:06 GMT
```

- 该字段值是long型，
- 是自格林尼治标准时间1970.1.1 00:00:00后的毫秒数
- 访问方法：
`Public long getIfModifiedSince()`
`Public void setIfModifiedSince(long ifModifiedSince)`

ifModifiedSince应用例子

演示Ex7-13,ch7.Last24.java 打印ifModifiedSince的默认值，将此值设置为24小时之前，并打印这个新值。然后下载并显示文档，但只是在最后24小时内有所修改时才会显示文档。

Last24.java

```
3 import java.io.*;
4 import java.net.*;
5 import java.util.*;
6 public class Last24 {
7     public static void main (String[] args) {
8         // Initialize a Date object with the current date and time
9         Date today = new Date();
10        long millisecondsPerDay = 24 * 60 * 60 * 1000;
```

ms



```
11
12     for (int i = 0; i < args.length; i++) {
13         try {
14             URL u = new URL(args[i]);
15             URLConnection uc = u.openConnection();
16             System.out.println("Original if modified since: "
17                 + new Date(uc.getIfModifiedSince()));
18             uc.setIfModifiedSince((new Date(today.getTime()
19                 - millisecondsPerDay)).getTime());
20             System.out.println("Will retrieve file if it's modified since "
21                 + new Date(uc.getIfModifiedSince()));
22             try (InputStream in = new BufferedInputStream(uc.getInputStream())) {
23                 Reader r = new InputStreamReader(in);
24                 int c;
25                 while ((c = r.read()) != -1) {
26                     System.out.print((char) c);
27                 }
28                 System.out.println();
29             }
30         } catch (IOException ex) {
31             System.err.println(ex);
32         }
33     }
34 }
35 }
```

Protected boolean useCaches

- 有些客户端（比如web浏览器）可以从本地缓存获取文档
- applet可以访问浏览器的缓存。
- 独立的应用程序可使用java.net.ResponseCache类
- 如果有缓存，那么由useCaches变量确定是否可以使用缓存
- useCahces变量为true表示将使用缓存，false表示不使用缓存，默认值是true
- 编程时，使用该变量对应的get和set方法访问此变量

在程序中禁用缓存，确保总是获取文档的最新版本。通过将useCaches设置为false来实现：

```
Try{  
    URL u = new URL("http://www.soucebot.com/");  
    URLConnection uc = u.openConnection();  
    uc.setUseCatches(false);  
    //读取文档...  
}catch(IOException ex){  
    System.err.println(ex);  
}
```



配置HTTP客户端请求头信息

- HTTP客户端向服务器发送一个请求行和一个首部。Web服务器根据这些信息向不同的客户端 提供不同的页面。
- 通过设置客户端请求头和服务器端响应头中的字段来控制请求格式和响应格式。
- 需在打开连接前设置字段
- 仅对HTTP协议适用， 设置方法：
 - `public void setRequestProperty(String name, String value)`
 - `public void addRequestProperty(String name, String value)`



配置HTTP客户端请求头信息

- Cookie是名-值对的集合。HTTP客户端和服务端之间使用cookie存储一些受限的持久信息。服务器使用HTTP响应头部向客户端发送一个cookie，之后，客户端请求此服务器的URL,都会在HTTP请求头部中包含此cookie字段：

Cookie: username=elharo; password=ACD0X9F23JJn6G; session=100678945

给定一个URLConnection对象uc,如何将这个cookie增加到连接？

```
uc.setRequestProperty("Cookie",  
"username=elharo; password=ACD0X9F23JJn6G; session=100678945");
```



5.向服务器写入数据

- 当在客户端程序中需要向URLConnection写入数据，例如使用POST向Web服务器提交表单，或使用PUT上传文件。
- 则使用URLConnection的getOutputStream()方法返回一个OutputStream，用来写入数据传送给服务器。
- 步骤：
 - `URLConnection uc = u.openConnection();`
 - `uc.setDoOutput(true);`// URLConnection默认不允许输出，设置为允许
 - `uc.getOutputStream();`
 - `//开始写...`



如何写？

- 得到OutputStream后，将它串链到缓冲类等方便写的类，
- 如缓冲类BufferedOutputStream或BufferedWriter，
- 写入类DataOutputStream或OutputStreamWriter

- Try{

```
URL u = new URL("http://www.somehost.com/cgi-bin/acgi");
//打开连接，准备post
URLConnection uc = u.openConnection();
uc.setDoOutput(true);

OutputStream raw = uc.getOutputStream();
OutputStream buffered = new BufferedOutputStream(raw);
OutputStreamWriter out = new OutputStreamWriter(buffered, "8859-1");
out.write("first=Julie&middle=&last=Harting&work=String+Quartet\r\n");
out.flush();
out.close();
```
- }catch(IOExceptionex){
- System.err.println(ex);
- }



POST请求

用POST发送请求与GET的区别是：

- (1) 先调用setDoOutput(true)。
- (2) 使用URLConnection的getOutputStream()方法写入查询字符串，而不是附加到URL（发送x-www-form-urlencoded码的字符串）。Java会缓冲写入输出流的所有数据，直到流关闭为止。

→ flush

- 一个客户端的POST请求和服务器的响应例子：
- telnet www.cafeaulait.org 80
- Trying 152.19.134.41...
- Connected to www.cafeaulait.org
- POST /books/jnp3/postquery.phtml HTTP/1.0
- Accept: text/plain
- Content-type: application/x-www-form-urlencoded
- Content-length: 63
- Connection: close
- Host: www.cafeaulait.org
- username=Elliotte+Rusty+Harold&email=elharo%40ibiblio%2eorg



服务器的响应

HTTP/1.1 200 OK

Date: Sat, 04 May 2013 13:27:24 GMT

Server: Apache

Content-Style-Type: text/css

Content-Length: 864

Connection: close

Content-Type: text/html; charset=utf-8

```
<html xmlns=http://www.w3.org/1999/xhtml>
```

```
  <head>
```

```
    <title>Query Results</title>
```

```
  </head>
```

```
  <body>
```

```
    <h1>Query Results</h1>
```

```
    <p>You submitted the following name/value pairs:</p>
```

```
    ...
```

```
  </body>
```

```
</html>
```



POST请求编程例子

- 编写程序，向一个HTTP服务器提交一个表单：使用URLConnection类和第5章的QueryString类提交(post)表单数据。向服务器资源发送名字“Elliotte Rusty Harold”和电子邮件地址elharo@biblio.org。服务器资源位于http://www.cafeaulait.org/books/jnp4/postquery.html，是一个表单测试器，可接受任何使用POST或GET方法的输入，并返回一个HTML页面，显示所提交的名和值。返回的数据是HTML

演示讲解示例代码：ch7.FormPoster.java



```
3 import java.io.*;
4 import java.net.*;
5 import ch5.QueryString;
6 public class FormPoster {
7
8     private URL url;
9     // from Chapter 5, Example 5-8
10    private ch5.QueryString query = new QueryString();
11
12    public FormPoster (URL url) {
13        if (!url.getProtocol().toLowerCase().startsWith("http")) {
14            throw new IllegalArgumentException(
15                "Posting only works for http URLs");
16        }
17        this.url = url;
18    }
19
20    public void add(String name, String value) {
21        query.add(name, value);
22    }
23
24    public URL getURL() {
25        return this.url;
26    }
```

```
27
28 public InputStream post() throws IOException {
29
30     // open the connection and prepare it to POST
31     URLConnection uc = url.openConnection();
32     uc.setDoOutput(true);
33     try (OutputStreamWriter out
34         = new OutputStreamWriter(uc.getOutputStream(), "UTF-8")) {
35
36         // The POST line, the Content-type header,
37         // and the Content-length headers are sent by the URLConnection.
38         // We just need to send the data
39         out.write(query.toString());
40         out.write("\r\n");
41         out.flush(); // 且记不能漏掉此句，否则不会发送任何数据
42     }
43
44     // Return the response
45     return uc.getInputStream();
46 }
47
```



```
47
48 public static void main(String[] args) {
49     URL url;
50     if (args.length > 0) {
51         try {
52             url = new URL(args[0]);
53         } catch (MalformedURLException ex) {
54             System.err.println("Usage: java FormPoster url");
55             return;
56         }
57     } else {
58         try {
59             url = new URL(
60                 "http://www.cafeaulait.org/books/jnp4/postquery.phtml");
61         } catch (MalformedURLException ex) { // shouldn't happen
62             System.err.println(ex);
63             return;
64         }
65     }
```

```
66
67     FormPoster poster = new FormPoster(url);
68     poster.add("name", "Elliotte Rusty Harold");
69     poster.add("email", "elharo@ibiblio.org");
70
71     try (InputStream in = poster.post()) {
72         // Read the response
73         Reader r = new InputStreamReader(in);
74         int c;
75         while((c = r.read()) != -1) {
76             System.out.print((char) c);
77         }
78         System.out.println();
79     } catch (IOException ex) {
80         System.err.println(ex);
81     }
82 }
83 }
```



QueryString.java

```
2 import java.io.UnsupportedEncodingException;
3 import java.net.URLEncoder;
4 public class QueryString {
5     private StringBuilder query = new StringBuilder();
6     public QueryString() {
7     }
8     public synchronized void add(String name, String value) {
9         query.append('&');
10        encode(name, value);
11    }
12    private synchronized void encode(String name, String value) {
13        try {
14            query.append(URLEncoder.encode(name, "UTF-8"));
15            query.append('=');
16            query.append(URLEncoder.encode(value, "UTF-8"));
17        } catch (UnsupportedEncodingException ex) {
18            throw new RuntimeException("Broken VM does not support UTF-8");
19        }
20    }
21    public synchronized String getQuery() {
22        return query.toString();
23    }
24    @Override
25    public String toString() {
26        return getQuery();
27    }
28 }
```

POST请求例子分析与测试

- FormPoster使用URLConnection类和QueryString类提交POST表单数据。
- 在构造函数中设置URL。
- 查询字符串用add()方法构建。
- Post()方法负责向服务器发送数据，返回包含服务器响应的输入流。
- Main()方法实现测试，可向位于<http://www.cafeaulait.org/books/jnp4/postquery.phtml>的资源发送名字“Elliote Rusty Harold”和电邮地址elharo@ibiblio.org。此Web资源是一个表单测试器，接受任何使用POST或GET方法的输入，并返回一个未解析的HTML页面，显示提交的名和值。

提交表单数据编程总结

- 1.确定要发送给服务器的程序的名-值对
- 2.编写接受和处理请求的服务器端程序。如果没有使用定制数据编码，可以使用普通的HTML表单和Web浏览器测试此程序
- 3.在java程序中创建一个查询字符串。字符串形式：
name1=value1&name2=value2&name3=value3 在增加到查询字符串之前，先将各个名和值传递到URLEncoder.encode()。
- 4.打开一个URLConnection，指向将接受数据的程序的URL
- 5.调用setDoOutput(true)设置doOutput为true
- 6.将查询字符串写入到URLConnection的OutputStream
- 7.关闭URLConnection的OutputStream
- 8.从URLConnection的InputStream读取服务器响应



猜测MIME媒体类型

- 异构的网络环境，并非所有协议和服务端都会使用标准MIME类型正确地指定所传输的文件类型。
- FTP协议比MIME标准出现的早，这些老协议如何处理？
- 有些使用了MIME的HTTP服务器不提供MIME首部，或者提供了不正确的首部。
- 我们在编程时需要确定数据的MIME类型。有两个方法可调用
- `public static String guessContentTypeFromName(String name)`
根据文件名猜测类型
- `public static String guessContentTypeFromStream(InputStream in)`
根据流中前几字节数据猜测内容类型（in必须支持标记，读取了前面的字节之后，可以返回流开始处）
优先采用第一个方法



URLConnection类

- Java.net.HttpURLConnection类是URLConnection类的抽象子类。
- 包含更多处理HTTP请求方法和HTTP响应码常量，如获得和设置请求方法、确定是否重定向、获得响应码和消息、确定是否使用了代理服务器
- 是抽象类,不能直接创建其对象,需使用强制转换方式:

```
URL u = new URL("http://lesswrong.com/");
```

```
URLConnection uc = u.openConnection();
```

```
HttpURLConnection http = (HttpURLConnection) uc;
```

- public void setRequestMethod(String method) throws ProtocolException
 - 设置请求方法: GET、POST、HEAD、PUT、DELETE、OPTIONS、TRACE
 - 需要全大写

URLConnection与URLConnection

- URLConnection与URLConnection都是抽象类，无法直接实例化对象。其对象主要通过URL的openconnection方法获得。
- openConnection方法只创建URLConnection或者URLConnection实例，并不进行真正的连接操作。并且，每次openConnection都将创建一个新的实例。
- openConnection不进行的连接操作的原因在于，可以在连接操作进行之前，对URLConnection或者URLConnection实例的某些属性进行设置，如设置超时值等。
- 无论URLConnection或者URLConnection实例，其getInputStream之类属于应用层的操作，都会调用connect操作。但是，connectTimeout与ReaderTimeout并不相同。有可能在已连接的情况下，仍然Reader超时

请求方法

- Web客户端与Web服务器联系时，发送的第一个内容是请求行。如：
- GET catalog/jfcnut/index.html HTTP/1.0
 - GET方法: Web客户端从服务器获取文件
 - POST方法: 客户端向表单提交数据发送请求并获得响应
 - PUT方法: 将文件上传到服务器
 - DELETE方法: 删除服务器的文件
 - HEAD方法: 只请求文档的首部
 - OPTIONS方法: 向服务器询问一个指定URL支持的选项列表
 - TRACE方法: 跟踪请求本身
- `public void setRequestMethod(String method) throws ProtocolException`
 - 设置请求方法: GET、POST、HEAD、PUT、DELETE、OPTIONS、TRACE
 - 需要全大写



HEAD方法

- 请求信息跟GET同，告诉服务器只返回HTTP头部，不用实际发送文件
- 服务器只返回头信息
- 此方法常用于检查文件在最后一次缓存之后是否有修改。
- 减少不必要信息的获取和传输，使程序高效
- 编程使用HEAD请求方法,获得和显示URL文件在服务器上的最后一次修改时间.
- 编程示例ch7.LastModified.java



```
1 package ch7;
2
3 import java.io.*;
4 import java.net.*;
5 import java.util.*;
6
7 public class LastModified {
8
9     public static void main(String[] args) {
10         for (int i = 0; i < args.length; i++) {
11             try {
12                 URL u = new URL(args[i]);
13                 HttpURLConnection http = (HttpURLConnection) u.openConnection();
14                 http.setRequestMethod("HEAD");
15                 System.out.println(u + " was last modified at "
16                     + new Date(http.getLastModified()));
17             } catch (MalformedURLException ex) {
18                 System.err.println(args[i] + " is not a URL I understand");
19             } catch (IOException ex) {
20                 System.err.println(ex);
21             }
22             System.out.println();
23         }
24     }
25 }
```

DELETE方法

- DELETE方法用于删除web服务器上位于指定URL的文件
- 该方法存在安全风险,不少服务器的配置可能不支持此方法.
- 通常要求完成身份认证才支持.
- 典型的DELETE请求:

DELETE /javafaq/2008march.html HTTP/1.1

Host: www.ibiblio.org

Accept: text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2

Connection: close



对服务器DELETE的响应

- 服务器可以拒绝此请求，或要求提供身份认证。
- 若接受请求，如何响应删除取决于服务器厂商的实现细节，可能会删除文件、或将文件移到回收站中、或将文件标记为不可读。
- 拒绝请求响应：

HTTP/1.1 405 Method Not Allowed

Date: Sat, 04 May 2013 13:22:12 GMT

Server: Apache

Allow: GET,HEAD,POST,OPTIONS,TRACE

Content-Length: 334

Connection: close

Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">

<html>

 <head>

 <title>405 Method Not Allowed</title>

 </head><body>

 ...

 </body>

</html>



PUT方法

- 此方法允许客户端将文档放在网站的抽象层次结构中，但不需要知道网站如何映射到实际的本地文件系统。
- PUT方法的使用通常需要身份认证，且服务器必须特别配置支持PUT方法，多数服务器不直接支持PUT方法
- 一个编辑器如何用PUT将一个文件存放在Web服务器上：

PUT /blog/wp-app.php/service/pomodoros.html HTTP/1.1

Host: www.elharo.com

Authorization: Basic ZGFmZnk6c2VjZXJldA==

Content-Type: application/atom+xml;type=entry

Content-Length: 329

If-Match: "e180ee84f0671b1"

<?xml version="1.0" ?>

<entry xmlns="http://www.w3.org/2005/Atom">

 <title>The Power of Pomodoros</title>

 <id>urn:uuid:1225c695-cfb8-4ebb-aaaa-80da344efa6a</id>

 <updated>2013-02-23T19:22:11Z</updated>

 <author><name>Elliotte Harold</name></author> <content>Until recently.... </content>

</entry>



OPTIONS方法

方法

- OPTIONS请求方法询问某个特定的URL支持哪些选项
- 如果请求的URL是星号(*),此请求将应用于整个服务器而不是服务器上的某个URL. 例如请求:

OPTIONS /xml/ HTTP/1.1

Host: www.ibiblio.org

Accept: text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2

Connection: close

- 服务器对OPTIONS请求的响应:

HTTP/1.1 200 OK

Date: Sat, 04 May 2013 13:52:53 GMT

Server: Apache

Allow: GET,HEAD,POST,OPTIONS,TRACE

Content-Style-Type: text/css

Content-Length: 0

Connection: close

Content-Type: text/html; charset=utf-8



TRACE方法

- TRACE方法会发送HTTP头部,服务器从客户端接收此HTTP头部,用于查看服务器和客户端之间的代理服务器做了哪些修改.

- TRACE请求:

TRACE /xml/ HTTP/1.1

Hello: Push me

Host: www.ibiblio.org

Accept: text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2

Connection: close

- 服务器对TRACE请求的响应

HTTP/1.1 200 OK

Date: Sat, 04 May 2013 14:41:40 GMT

Server: Apache

Connection: close

Content-Type: message/http

TRACE /xml/ HTTP/1.1

Hello: Push me

Host: www.ibiblio.org

Accept: text/html, image/gif, image/jpeg, *; q=.2, */*; q=.2

Connection: close

前5行是服务器正常的响应HTTP头部

后面是原客户端请求的回显.若存在代理,可能回显与原请求不一样

断开与服务器的连接

- HTTP1.1支持持久连接,使用Keep-Alive时, 服务器不会因为已经向客户端发送了最后一个字节的数据就立即关闭连接。
- 最好由客户端在确认工作结束时关闭连接
- 在服务器关闭连接之前, 如果再次连接同一个服务器, HttpURLConnection类会重用socket。
- 客户端需调用方法显式断开连接:
- `public void HttpURLConnection.disconnect()`
- 若连接上有打开的流:
 - 断开连接可以关闭流
 - 关闭流不会断开连接



处理服务器的响应

■ 不同的响应

200 OK响应，
表示所请求的
文档已找到

```
HTTP/1.1 200 OK
Cache-Control: max-age=3, must-revalidate
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
Date: Sat, 04 May 2013 14:01:16 GMT
Keep-Alive: timeout=5, max=200
Server: Apache
Transfer-Encoding: chunked
Vary: Accept-Encoding, Cookie
WP-Super-Cache: Served supercache file from PHP

<HTML>
<HEAD>
rest of document follows...
```

404 NotFound响应，表示
所请求的URL不再指向一个文档

```
HTTP/1.1 404 Not Found
Date: Sat, 04 May 2013 14:05:43 GMT
Server: Apache
Last-Modified: Sat, 12 Jan 2013 00:19:15 GMT
ETag: "375933-2b9e-4d30c5cb0c6c0;4d02eaff53b80"
Accept-Ranges: bytes
Content-Length: 11166
Connection: close
Content-Type: text/html; charset=ISO-8859-1

<html>
<head>
```

```
<title>Lost ... and lost</title>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
</head>
<body bgcolor="#FFFFFF">
  <h1>404 FILE NOT FOUND</h1>
  Rest of error message follows...
```

```
HTTP/1.1 301 Moved Permanently
Connection: Keep-Alive
Content-Length: 299
Content-Type: text/html; charset=iso-8859-1
Date: Sat, 04 May 2013 14:20:58 GMT
Keep-Alive: timeout=5, max=200
Location: http://www.cafeaulait.org/
Server: Apache
```

301 响应码表示所请求的资源已经永久移动到一个新位置，浏览器应当重定向到这个新位置，并更新所有指向老位置的书签。

处理服务器的响应

- 获得响应码:
- **public int getResponseCode() throws IOException**
 - 获取响应码
- 响应码后面的文本字符串称为响应消息 (response message)
- **public String getResponseMessage() throws IOException**
 - 获取响应消息
- 默认情况下, HttpURLConnection会跟随重定向, 遇到300级响应码时, 浏览器自动从新位置加载文档。为防止安全风险, 可以调用方法确定是否跟随重定向:
- **public static boolean getFollowRedirects()**
 - 返回true表示跟随重定向, 否则返回false
- **public static void setFollowRedirects(boolean follow)**
 - 设置为true表示允许HttpURLConnection对象跟随重定向, 设置为false表示阻止跟随。

处理服务器的响应

- HttpURLConnection类包括36个命名常量，表示常见的响应码
 - ▣ 例如 HttpURLConnection.OK,
 - ▣ 例如 HttpURLConnection.NOT_FOUND
- 演示例子程序ch7.SourceViewer3.java， 修改了之前的源码查看程序，可以查看响应消息。



```
2 import java.io.*;
3 import java.net.*;
4 public class SourceViewer3 {
5     public static void main (String[] args) {
6         for (int i = 0; i < args.length; i++) {
7             try {
8                 // Open the URLConnection for reading
9                 URL u = new URL(args[i]);
10                HttpURLConnection uc = (HttpURLConnection) u.openConnection();
11                int code = uc.getResponseCode();
12                String response = uc.getResponseMessage();
13                System.out.println("HTTP/1.x " + code + " " + response);
14                for (int j = 1; ; j++) {
15                    String header = uc.getHeaderField(j);
16                    String key = uc.getHeaderFieldKey(j);
17                    if (header == null || key == null) break;
18                    System.out.println(uc.getHeaderFieldKey(j) + ": " + header);
19                }
20                System.out.println();
21
22                try (InputStream in = new BufferedInputStream(uc.getInputStream())) {
23                    // chain the InputStream to a Reader
24                    Reader r = new InputStreamReader(in);
25                    int c;
26                    while ((c = r.read()) != -1) {
27                        System.out.print((char) c);
28                    }
29                }
30            } catch (Exception e) {
31                System.out.println(e.getMessage());
32            }
33        }
34    }
35 }
```

处理服务器的响应

SourceViewer3.java

```
29     }
30     } catch (MalformedURLException ex) {
31         System.err.println(args[0] + " is not a parseable URL");
32     } catch (IOException ex) {
33         System.err.println(ex);
34     }
35 }
36 }
37 }
```



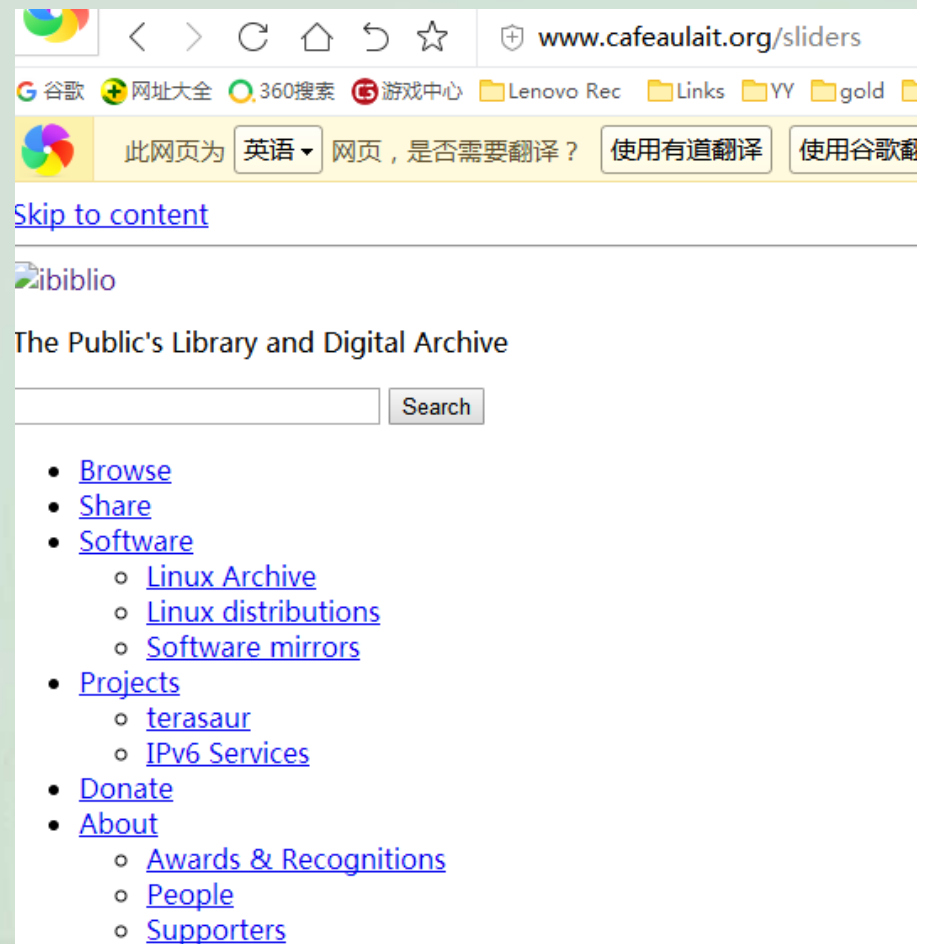
错误流的读取

- 服务器遇到一个错误，仍会在消息主体中返回有用的信息
- 例如客户端向网站请求的页面<http://www.cafeaulait.org/sliders>不存在时，服务器返回404错误码的同时，会发送一个搜索页面，帮助用户确定缺少的页面可能在哪里。

Public InputStream getErrorStream()
getInputStream失败后，
可在catch块中调用此方法获得错误流。

演示编码例子：

ch7.SourceView4.java，程序从输入流读取数据，若读取失败，则读取错误流。



SourceViewer4.java

```
3 import java.io.*;
4 import java.net.*;
5 public class SourceViewer4 {
6
7     public static void main (String[] args) {
8         try {
9             URL u = new URL(args[0]);
10            HttpURLConnection uc = (HttpURLConnection) u.openConnection();
11            try (InputStream raw = uc.getInputStream()) {
12                printFromStream(raw);
13            } catch (IOException ex) {
14                printFromStream(uc.getErrorStream());
15            }
16        } catch (MalformedURLException ex) {
17            System.err.println(args[0] + " is not a parseable URL");
18        } catch (IOException ex) {
19            System.err.println(ex);
20        }
21    }
22
23    private static void printFromStream(InputStream raw) throws IOException {
24        try (InputStream buffer = new BufferedInputStream(raw)) {
25            Reader reader = new InputStreamReader(buffer);
26            int c;
27            while ((c = reader.read()) != -1) {
28                System.out.print((char) c);
29            }
30        }
31    }
32 }
```


其它参考资源

- URLConnection类

<http://download.oracle.com/technetwork/java/javase/6/docs/zh/api/java/net/URLConnection.html>



利用URLConnection对象和Internet交互

- 从Internet获取网页
- 发送请求,将网页以流的形式读回来.
 - 1)创建一个URL对象:`URL url = new URL("http://www.sohu.com");`
 - 2)利用URLConnection对象从网络中获取网页数据:`URLConnection conn = (URLConnection) url.openConnection();`
 - 3)设置连接超时:`conn.setConnectTimeout(6* 1000);`
 - 4)对响应码进行判断:`if (conn.getResponseCode() != 200)`
`throw new RuntimeException("请求url失败");`
 - 5)得到网络返回的输入流:`InputStream is = conn.getInputStream();`
 - 6)`String result = readData(is, "GBK");`
`conn.disconnect();`



利用URLConnection对象和Internet交互

- 从Internet获取文件
- 利用URLConnection对象,我们可以从网络中获取文件数据
- 1)创建URL对象,并将文件路径传入:
`URL url = new URL("http://photocdn.sohu.com/20100125/lmg269812337.jpg");`
- 2)创建URLConnection对象,从网络中获取文件数据:
`URLConnection conn = (URLConnection) url.openConnection();`
- 3)设置连接超时:
`conn.setConnectTimeout(6* 1000);`
- 4)对响应码进行判断:
`if (conn.getResponseCode() != 200) throw new RuntimeException("请求url失败");`
- 5)得到网络返回的输入流:
`InputStream is = conn.getInputStream();`
- 6)将得到的文件流写出:
`outStream.write(buffer, 0, len);`



利用URLConnection对象和Internet交互

■ 向Internet发送请求参数

- 1)将地址和参数存到byte数组中:`byte[] data = params.toString().getBytes();`
- 2)创建URL对象:`URL realUrl = new URL(requestUrl);`
- 3)通过URLConnection对象,向网络地址发送请求:`URLConnection conn = (URLConnection) realUrl.openConnection();`
- 4)设置容许输出:`conn.setDoOutput(true);`
- 5)设置不使用缓存:`conn.setUseCaches(false);`
- 6)设置使用POST的方式发送:`conn.setRequestMethod("POST");`
- 7)设置维持长连接:`conn.setRequestProperty("Connection", "Keep-Alive");`
- 8)设置文件字符集:`conn.setRequestProperty("Charset", "UTF-8");`
- 9)设置文件长度:`conn.setRequestProperty("Content-Length", String.valueOf(data.length));`
- 10)设置文件类型:`conn.setRequestProperty("Content-Type", "application/x-www-form-urlencoded");`
- 11)以流的方式输出.
- 总结:--发送POST请求必须设置允许输出
- --不要使用缓存,容易出现问题.
- --在开始用URLConnection对象的setRequestProperty()设置,就是生成HTML文件头

小结

- 重难点
- URLConnection及其应用
- HttpURLConnection及其应用



谢谢

